

Media Playback SDK

Project explained in simple terms: summary, components, workflows and architecture

Resume line being explained

“Led technical direction, system architecture, and API design for a C++/Objective-C media playback SDK adopted by 4 third-party partners, cutting integration time 45% by aligning requirements with partner organizations and shipping versioned APIs and reference players.”

Contents

1. The Big Picture (Simple Summary)

- 1.1 What was built
- 1.2 Who used it
- 1.3 What my role was
- 1.4 Why it mattered: integration time cut by 45%

2. Key Terms in Plain English

3. Architecture Diagram

- 3.1 How to read the diagram
- 3.2 Layer-by-layer walkthrough
- 3.3 Delivery and governance diagram

4. Every Component Explained

- 4.1 Partner apps and reference players [1]–[6]
- 4.2 Public, versioned API [7]–[10]
- 4.3 Objective-C++ bridge [11]–[13]
- 4.4 Core engine: control [14]–[17]
- 4.5 Core engine: fetch and decide [18]–[22]
- 4.6 Core engine: media pipeline [23]–[27]
- 4.7 Apple platform frameworks [28]–[33]
- 4.8 Outside services [34]–[36]
- 4.9 Delivery and governance [37]–[42]

5. Workflows, Step by Step

- 5.1 A: Aligning requirements with partners
- 5.2 B: Designing and versioning an API
- 5.3 C: Build, test and release
- 5.4 D: How a partner integrates the SDK
- 5.5 E: Runtime playback, end to end
- 5.6 Player states and allowed moves
- 5.7 F: Error handling
- 5.8 G: Upgrading to a new SDK version

6. Results and Why They Happened

- 6.1 How to explain this in an interview

1. The Big Picture (Simple Summary)

In one sentence: I designed and led the building of a ready-made “video player engine” that other companies could drop into their Apple apps, and I made it so easy to plug in that partners finished their integration almost twice as fast as before.

1.1 What was built

Think of a car engine sold to car makers. Each car maker designs its own body, seats and dashboard, but they all want a reliable engine that just works. This project is the **engine for playing video and audio** inside iPhone, iPad, Mac (and similar) apps. It is called an **SDK (Software Development Kit)**: a package of code, documentation and examples that other developers use to build their own products.

The engine is written in **C++** (a fast language used for heavy work like decoding video) and wrapped in **Objective-C** (Apple’s traditional app language, which Swift apps can also use). So partner apps talk to it in a language they already know, while the heavy lifting runs in fast C++ underneath.

1.2 Who used it

Four outside companies (third-party partners) adopted the SDK in their own apps. Each had its own needs, such as different video formats, copy-protection (DRM), analytics or custom player screens.

1.3 What my role was

- **Technical direction:** deciding what to build, in what order, and which technologies to use.
- **System architecture:** designing how the pieces fit together (the layers and boxes in the diagram in Section 3).
- **API design:** designing the “buttons and dials” partners use, for example `play()`, `pause()`, `seek(to:)`, and the events the player reports back.

1.4 Why it mattered: integration time cut by 45%

“Integration time” is how long it takes a partner team to go from receiving the SDK to having it working inside their app. It dropped by 45% (for example, from about 10 weeks to about 5.5 weeks). Three things made that happen:

1. **Aligning requirements with partners early:** agreeing up front what each partner needed, so there were fewer surprises and rework later.
2. **Versioned APIs:** clearly numbered versions (v1, v2, v3) with promises about what will and won’t change, so partners could upgrade safely without their app breaking.

3. **Reference players:** complete, working sample apps that show exactly how to use the SDK. Partners could copy patterns instead of figuring everything out themselves.

Note: The resume line does not list every internal detail. This document describes a standard, realistic architecture for an SDK of this kind. Rename components to match your actual project before using it in interviews.

2. Key Terms in Plain English

Read this first. Every technical word used later is explained here.

Term	Simple meaning
SDK	A toolkit (code library + docs + examples) that other developers add to their app to get a feature, here video playback.
API	The menu of commands and messages the SDK offers, e.g. <code>play()</code> , <code>pause()</code> , and “video finished” notifications.
C++	A fast, low-level programming language. Used for the engine’s heavy work; can also run on non-Apple platforms.
Objective-C / Objective-C++	Apple’s classic app language. Objective-C++ files (.mm) can mix Objective-C and C++, so they act as the translator.
Swift	Apple’s modern app language. It can call Objective-C APIs directly, so partners using Swift could use the SDK too.
Streaming (HLS / DASH)	Video is cut into small chunks (2–10 seconds each) delivered over the internet. HLS is Apple’s format; DASH is an industry standard.
Manifest / playlist	A small text file listing all chunks and available quality levels, like a table of contents for the video.
Segment	One small chunk of video or audio downloaded at a time.
Bitrate / rendition	A quality level. Higher bitrate means sharper picture but needs a faster internet connection.
ABR (Adaptive Bitrate)	Automatically switching quality up or down as internet speed changes, to avoid freezing.
Demuxing	Separating a downloaded chunk into its audio, video and subtitle parts (like separating a sandwich into its layers).
Decoding	Turning compressed video/audio data into actual pictures and sound samples.
Rendering	Drawing the decoded pictures on the screen at the right moment.

Term	Simple meaning
DRM	Digital Rights Management, i.e. copy protection. The video is encrypted and needs a key (license) to play.
FairPlay	Apple's DRM system.
CDN	Content Delivery Network: servers around the world that store video chunks close to viewers for fast download.
Buffer	Video downloaded ahead of time and waiting to be played, like water in a tank.
State machine	A strict list of situations the player can be in (Idle, Playing, Paused...) and the allowed moves between them.
Semantic Versioning (SemVer)	Version numbers like 2.4.1 = MAJOR.MINOR.PATCH. Major = may break things; Minor = new features, nothing breaks; Patch = bug fixes only.
XCFramework	Apple's package format for shipping a compiled library that works on devices, simulators and Macs in one bundle.
Reference player	A complete, official sample app showing the correct way to use the SDK.
QoE	Quality of Experience: measures like start-up time, how often video freezes, and average picture quality.
ARC / shared_ptr	Automatic memory management in Objective-C (ARC) and in C++ (shared_ptr). The bridge keeps both in agreement.

3. Architecture Diagram

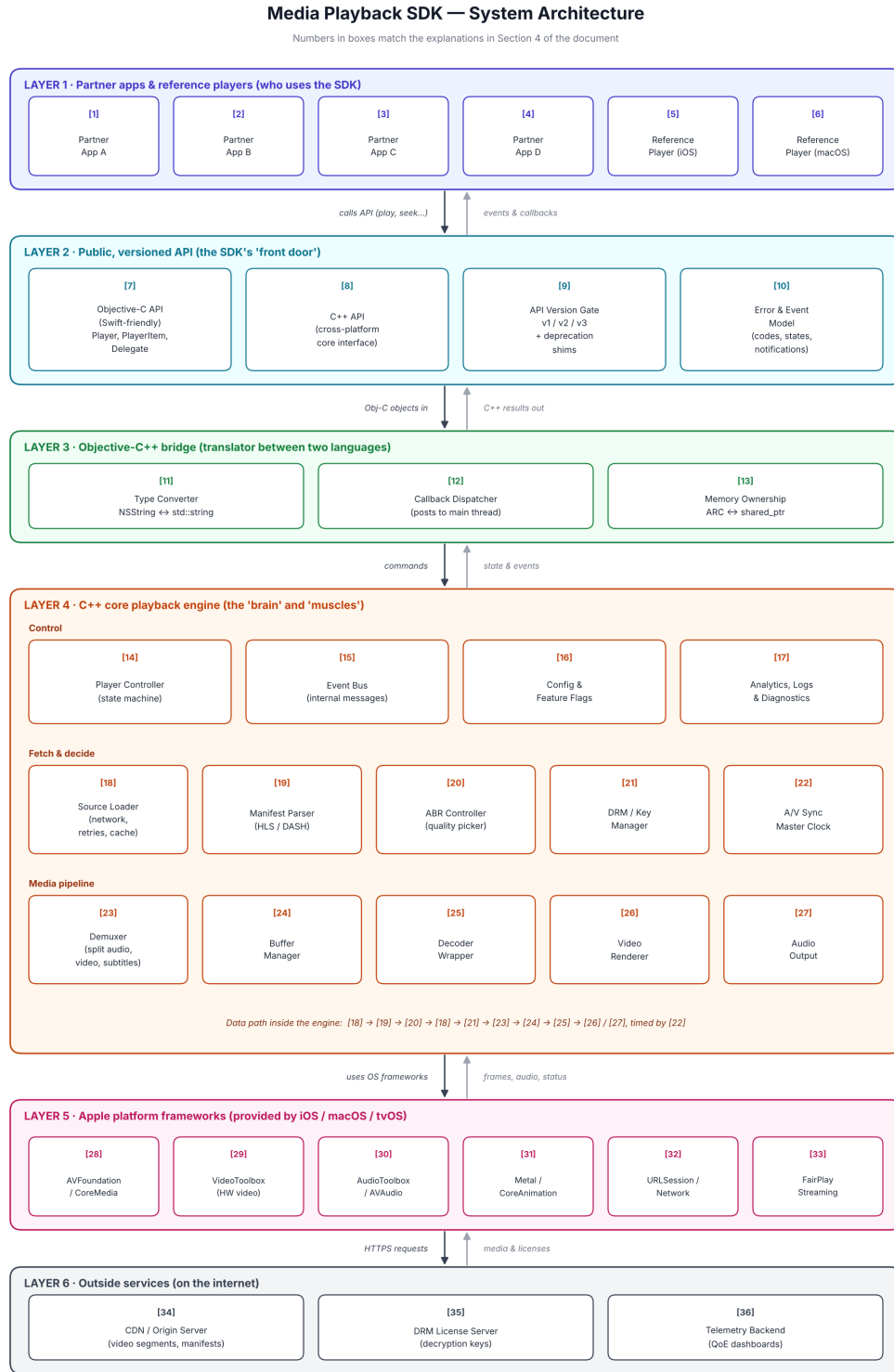


Figure 1. Layered architecture of the media playback SDK (boxes [1] to [36]).

Figure 1 above shows the whole system as six stacked layers. A request (like “play”) travels **downward** from the partner app to the internet. Results (video frames, events, errors) travel **upward** back to the app. Each box has a number in square brackets; Section 4 explains every numbered box in detail.

3.1 How to read the diagram

- **Coloured bands = layers.** Each layer only talks to the layer directly above or below it. This keeps the design clean: changing one layer does not break the others.
- **Dark down-arrows** = commands and requests going down (e.g. “play”, “download this chunk”).
- **Grey up-arrows** = answers coming back up (e.g. video frames, “now playing”, error messages).
- **Numbers [1]–[36]** match the component explanations in Section 4. Numbers [37]–[42] belong to Figure 2 (delivery process).
- **The italic line inside Layer 4** shows the order in which video data flows through the engine.

3.2 Layer-by-layer walkthrough

Layer	What it is (simple)	Talks to	Arrow meaning
1. Partner apps & reference players	The apps that use the SDK: 4 partner apps and 2 official sample apps.	Layer 2 only	Down: call API methods. Up: receive events and callbacks.
2. Public, versioned API	The SDK’s “front door”: the only part partners see and depend on.	Layer 1 and Layer 3	Down: Objective-C objects passed in. Up: results converted from C++.
3. Objective-C++ bridge	A translator converting between Objective-C and C++ data, threads and memory.	Layer 2 and Layer 4	Down: commands. Up: state changes and events.
4. C++ core engine	The brain (control) and muscles (download, decode, display) of playback.	Layer 3 and Layer 5	Down: uses Apple frameworks. Up: frames, audio and status.
5. Apple platform frameworks	Ready-made system libraries from Apple for hardware decoding, drawing, sound, networking and DRM.	Layer 4 and Layer 6	Down: HTTPS requests. Up: media data and licenses.

Layer	What it is (simple)	Talks to	Arrow meaning
6. Outside services	Servers on the internet: video storage (CDN), DRM license server, and analytics backend.	Layer 5	Serve video, keys, and receive telemetry.

3.3 Delivery and governance diagram

Figure 2 shows the **process side** of the project: how requirements, API design, testing and releases were organised. This process is where most of the 45% integration-time saving came from.

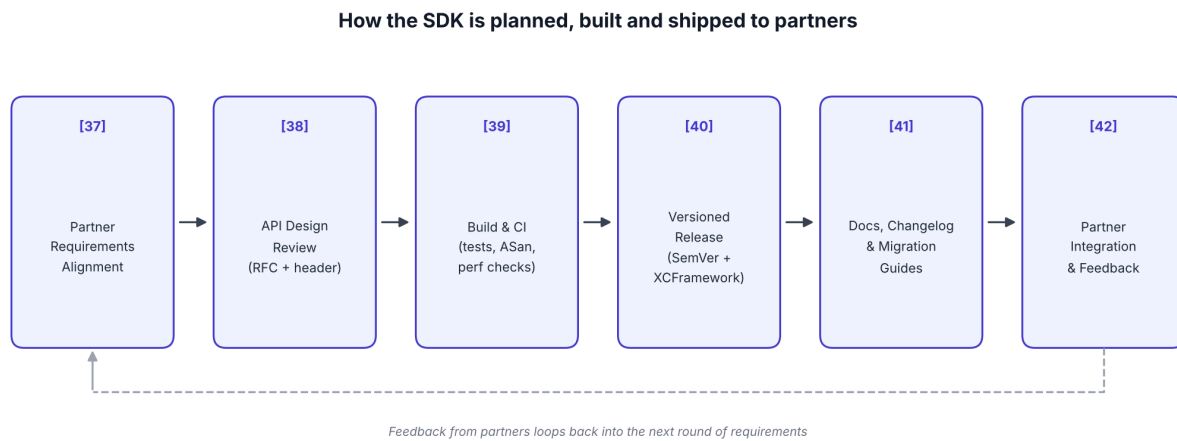


Figure 2. Planning-to-delivery pipeline (boxes [37] to [42]), with the partner feedback loop.

- **Solid arrows** show the order of the stages, left to right.
- **Dashed grey arrow** is the feedback loop: what partners learn during integration becomes input for the next round of requirements.

4. Every Component Explained

Each component below has a small fact card. **What it is** gives a plain description, **Analogy** gives an everyday comparison, **What it does** lists its jobs, **Inputs / Outputs** say what goes in and comes out, and **Fine details** cover the deeper engineering points.

4.1 Layer 1: Partner apps and reference players

[1–4] Partner Apps A, B, C and D

What it is	The real products of the four outside companies (for example a streaming service, a news app, an e-learning app and a sports app) that embed the SDK.
Analogy	Four different car brands that all buy the same engine.
What it does	Builds its own screens, buttons, branding and business logic, and calls the SDK whenever it needs to play media.
Inputs / Outputs	Input: user taps (play, pause, seek). Output: API calls to Layer 2. Receives events such as “buffering” or “ended” to update its screens.
Fine details	Each partner had different priorities, e.g. DRM-protected movies, live sports with low delay, offline downloads, or custom analytics. Partners integrate the SDK through Swift Package Manager or CocoaPods, or by dragging in the XCFramework. They should only depend on the public API [7]/[8], never on internal code, so the SDK can evolve safely.
Why it matters	These four adoptions are the business result of the project; their needs drove the requirements process [37].

[5] Reference Player (iOS)

What it is	An official, fully working sample iPhone/iPad app built by our team using only the public API.
Analogy	A show-home: a finished house that shows buyers exactly how everything should look and work.

What it does	Shows the correct way to create the player, load a video, handle events, show controls, support Picture-in-Picture, AirPlay and background audio, and handle errors. Serves as a live test that the public API is complete and pleasant to use (“eating our own cooking”).
Inputs / Outputs	Input: sample streams (clear and DRM-protected). Output: copy-paste-ready code patterns for partners.
Fine details	Written in Swift and Objective-C so both kinds of partner teams have an example. Each SDK release ships with an updated reference player version that matches it. Includes a debug overlay showing bitrate, buffer level and dropped frames, which partners used for troubleshooting.
Why it matters	A major driver of the 45% cut: partners started from working code instead of a blank page.

[6] Reference Player (macOS)

What it is	The same idea as [5], for Mac apps (and a base for tvOS).
Analogy	The same show-home, but built for a different neighbourhood.
What it does	Demonstrates desktop-specific behaviour such as window resizing, full-screen mode, keyboard shortcuts and multiple displays.
Inputs / Outputs	Same as [5].
Fine details	Shares most code with [5]; only the user interface layer differs (AppKit instead of UIKit). This proves the SDK API is truly cross-platform within Apple devices.
Why it matters	Partners shipping on Mac had a ready example too, so no partner was left without a starting point.

4.2 Layer 2: Public, versioned API

[7] Objective-C API (Swift-friendly)

What it is	The main set of classes partners use: e.g. <code>MPPlayer</code> (controls playback), <code>MPPlayerItem</code> (describes one video), <code>MPPlayerView</code> (shows the picture) and <code>MPPlayerDelegate</code> (receives events).
Analogy	The dashboard of a car: steering wheel, pedals and gauges. The driver never touches the engine directly.
What it does	Offers simple commands: load, play, pause, seek, set volume, choose audio track or subtitles. Exposes read-only status: current time, duration, state, buffered range. Sends events back through a delegate or block callbacks.
Inputs / Outputs	Input: calls from the partner app. Output: forwarded commands to the bridge [11]–[13]; events back to the app.
Fine details	Uses nullability annotations and <code>NS_SWIFT_NAME</code> so it looks natural in Swift (e.g. <code>player.seek(to:)</code>). Thread-safe: methods can be called from any thread; callbacks always arrive on the main thread. Small surface on purpose: fewer, well-named methods are easier to learn and to keep stable. Errors are returned as <code>NSError</code> with documented domain and codes from [10].
Why it matters	This is what partners actually learn. A clean, familiar API is the single biggest factor in integration speed.

[8] C++ API (cross-platform core interface)

What it is	A pure C++ interface to the same engine, without any Apple-specific types.
Analogy	A universal plug: the same engine can be connected to other “cars” (other platforms) later.
What it does	Lets C++-based partners or other platforms (e.g. a game engine or a future Android/desktop port) use the engine directly.

Inputs / Outputs	Input: C++ calls with <code>std::string</code> , <code>std::function</code> callbacks, etc. Output: commands to the core engine [14].
Fine details	Uses abstract interfaces (pure virtual classes) and factory functions so internals can change without recompiling partner code. Careful ABI stability: no STL types crossing binary boundaries in risky ways, using opaque handles (the “pimpl” pattern). The Objective-C API [7] is itself built on top of this same C++ core, so both stay in sync.
Why it matters	Keeps the engine portable and the architecture honest: platform-specific code stays in the outer layers.

[9] API Version Gate and Deprecation Shims

What it is	The rules and code that manage API versions (v1, v2, v3) so old partner code keeps working when the SDK is upgraded.
Analogy	A power adapter that lets an old plug fit a new socket.
What it does	Follows Semantic Versioning: breaking changes only in a MAJOR version; MINOR adds features; PATCH fixes bugs. When a method is replaced, the old one is marked deprecated (with a compiler warning naming the new method) but still works for at least one major version. Shims translate old calls into new ones internally.
Inputs / Outputs	Input: calls using old or new API names. Output: one consistent call into the engine.
Fine details	Uses availability macros (<code>API_AVAILABLE</code>, <code>API_DEPRECATED_WITH_REPLACEMENT</code>) so Xcode guides partners automatically. An automated API-diff check in CI [39] fails the build if a public header changes in a breaking way without a major version bump. Each release lists deprecations and removal dates in the changelog [41].
Why it matters	Partners could upgrade in hours instead of weeks, a key part of the “versioned APIs” promise on the resume.

[10] Error and Event Model

What it is	One shared, documented list of player states, event types and error codes.
Analogy	The warning lights on a car dashboard, each with a meaning written in the manual.
What it does	Defines states: Idle, Loading, Ready, Playing, Paused, Buffering, Seeking, Ended, Failed. Defines events: state changed, time update, bitrate changed, buffer empty/full, track list changed, error. Defines error codes grouped by area: network, manifest, DRM, decode, and internal.
Inputs / Outputs	Input: internal events from the event bus [15]. Output: clear, stable messages to partner apps.
Fine details	Each error says whether it is recoverable (the SDK will retry) or fatal (the app must act). Errors carry an underlying cause chain for debugging, e.g. DRM error caused by HTTP 403 from the license server. Codes never change meaning between versions; new codes are only added.
Why it matters	Partners can show correct messages to users and debug faster, without guessing.

4.3 Layer 3: Objective-C++ bridge**[11] Type Converter**

What it is	Code (in .mm files) that converts data between Objective-C types and C++ types.
Analogy	A currency exchange desk: dollars in, euros out, at an exact rate.
What it does	Converts NSString ↔ std::string (UTF-8), NSURL ↔ URL string, NSDictionary ↔ C++ config structs, NSError ↔ C++ error objects, CMTIME ↔ microseconds.
Inputs / Outputs	Input: Objective-C objects from [7]. Output: C++ values for the engine; and the reverse on the way up.

Fine details	Converts at the boundary only once, so the C++ engine never sees Apple types and stays portable.
Why it matters	Prevents a whole class of subtle bugs (text encoding, time rounding) from reaching partners.

[12] Callback Dispatcher

What it is	The piece that delivers engine events to the partner app on the correct thread.
Analogy	A receptionist who collects messages from the back office and hands them to the front desk one at a time.
What it does	Receives events from C++ background threads. Posts them to the main thread (where all iOS/macOS user interface work must happen). Coalesces very frequent events (e.g. time updates) to avoid flooding the app.
Inputs / Outputs	Input: C++ events. Output: delegate/block calls on the main queue.
Fine details	Uses Grand Central Dispatch (dispatch_async to the main queue) and weak references so a callback never reaches a player the app has already released.
Why it matters	Partners never get crashes from updating UI on the wrong thread, a common integration pitfall that the SDK removes.

[13] Memory Ownership Manager

What it is	Rules and helper code that keep Objective-C memory management (ARC) and C++ memory management (shared_ptr / unique_ptr) in agreement.
Analogy	A shared calendar for two roommates, so nobody throws something out while the other is still using it.
What it does	Each Objective-C wrapper owns its C++ object; when the wrapper is freed, the engine object is shut down safely, and the reverse cycle is broken with weak pointers.

Inputs / Outputs	Input: object creation and release. Output: no leaks and no use-after-free.
Fine details	Tested with Address Sanitizer and Instruments (Leaks, Allocations) in CI [39]; strict rule that no raw C++ pointer is stored in Objective-C objects.
Why it matters	Memory bugs are the hardest for partners to debug; preventing them inside the SDK saves partner time and builds trust.

4.4 Layer 4: C++ core engine, control group

[14] Player Controller (state machine)

What it is	The central coordinator: it decides what the player should do next.
Analogy	An orchestra conductor: it does not play an instrument, but tells everyone when to start and stop.
What it does	Holds the current state and only allows valid moves (see table in Section 5.6). Turns commands like “play” or “seek” into instructions for the loader, buffer, decoder and renderer. Reacts to internal events (buffer empty → Buffering; end of stream → Ended).
Inputs / Outputs	Input: commands from the bridge and events from [15]. Output: instructions to every engine component; state changes to [10].
Fine details	Commands are placed in a serial command queue so they run one at a time, in order, avoiding race conditions. Invalid commands (e.g. seek while Idle) return a clear error instead of crashing. Owens the lifetime of all other engine components.
Why it matters	Predictable behaviour: partners get the same reaction to the same action every time.

[15] Event Bus

What it is	An internal message system that components use to tell each other what happened.
-------------------	--

Analogy	An office notice board: anyone can post; anyone interested reads.
What it does	Carries events like “segment downloaded”, “buffer low”, “decode error” and “bitrate changed” between components without them needing to know each other.
Inputs / Outputs	Input: events from any component. Output: delivery to subscribers (controller, analytics, bridge).
Fine details	Publish/subscribe design with typed events; delivery on a dedicated engine thread; keeps components loosely coupled so they can be replaced or tested alone.
Why it matters	Makes the engine easier to extend (e.g. adding analytics) without touching playback code.

[16] Config and Feature Flags

What it is	Central settings that tune the engine, with switches to turn features on or off.
Analogy	The settings menu on a TV.
What it does	Holds values like buffer target (e.g. 30 seconds), max resolution, start-up bitrate, retry counts, timeouts. Lets each partner choose options without code changes, e.g. enable low-latency live mode.
Inputs / Outputs	Input: partner configuration via the API, plus defaults. Output: values read by every component.
Fine details	New, risky features ship behind flags (off by default) so one partner can trial them while others stay on stable behaviour.
Why it matters	One SDK build fits four partners with different needs, avoiding custom forks.

[17] Analytics, Logs and Diagnostics

What it is	The engine’s “flight recorder” and health reporter.
-------------------	---

Analogy	A fitness tracker for the video player.
What it does	Measures QoE: time to first frame, rebuffering count and duration, average bitrate, dropped frames, errors. Writes structured logs with levels (error, warning, info, debug). Optionally sends metrics to the telemetry backend [36] or hands them to the partner's own analytics.
Inputs / Outputs	Input: events from [15]. Output: metrics, logs and a debug overlay in the reference players.
Fine details	Privacy first: no personal data collected; partners can opt out; logs can be exported to attach to bug reports.
Why it matters	Partners and our team could find problems in minutes rather than days.

4.5 Layer 4: C++ core engine, fetch and decide group

[18] Source Loader (network, retries, cache)

What it is	The downloader for manifests, video segments and keys.
Analogy	A delivery driver who fetches parcels and tries again if nobody answers.
What it does	Downloads files over HTTPS using [32]. Retries failed downloads with exponential backoff (wait 1s, 2s, 4s...). Measures download speed and reports it to the ABR controller [20]. Can cache segments for fast seeking or offline playback.
Inputs / Outputs	Input: URLs from [19] and [20]. Output: raw bytes to [19], [21] and [23]; speed samples to [20].
Fine details	Supports parallel downloads, request cancellation on seek, custom HTTP headers/tokens from partners, and switching to backup CDN URLs on failure.
Why it matters	Network is the most common failure; good retries mean users rarely see errors.

[19] Manifest Parser (HLS / DASH)

What it is	Reads the playlist file that describes the video.
Analogy	Reading a book's table of contents to know which chapters exist and where they are.
What it does	Extracts the list of quality levels (renditions), audio languages, subtitle tracks, segment URLs and durations, and DRM information.
Inputs / Outputs	Input: manifest text (.m3u8 for HLS, .mpd for DASH). Output: a structured C++ model of the presentation.
Fine details	For live streams it re-downloads the manifest periodically to find new segments; it validates the file and reports clear manifest errors via [10].
Why it matters	Supporting both formats let partners with different backends use the same SDK.

[20] ABR Controller (quality picker)

What it is	The logic that chooses which quality level to download next.
Analogy	Cruise control that slows down on hills (slow internet) and speeds up on flat roads.
What it does	Combines measured bandwidth (from [18]) and buffer level (from [24]) to choose the best safe bitrate. Switches down quickly when the network drops; switches up carefully to avoid ping-ponging. Respects limits such as screen size and partner caps (e.g. max 1080p on cellular).
Inputs / Outputs	Input: bandwidth samples, buffer level, config [16]. Output: the next segment URL for [18].
Fine details	Uses a hybrid throughput + buffer-based algorithm with smoothing (moving averages) and safety margins; the algorithm is pluggable so it can be tuned per partner.
Why it matters	Fewer freezes and sharper video: the two things viewers notice most.

[21] DRM / Key Manager

What it is	Handles copy-protected content.
Analogy	A doorman who checks your ticket with the box office before letting you in.
What it does	Detects encrypted content from the manifest. Asks FairPlay [33] for a license request, sends it to the license server [35] (through a partner hook to add auth tokens), and installs the returned key. Renews or refreshes keys for long or live streams; supports persistent keys for offline playback.
Inputs / Outputs	Input: DRM info from [19]. Output: decryption keys used by the decode path.
Fine details	Keys never leave Apple's secure system; the SDK only passes opaque data. Clear errors distinguish "not entitled", "license server unreachable" and "device not supported".
Why it matters	Premium video partners cannot launch without DRM; a ready-made flow saved them weeks.

[22] A/V Sync Master Clock

What it is	The single source of "what time is it in the video".
Analogy	A metronome that keeps every musician in the same beat.
What it does	Keeps lips moving in time with speech by timing video frames against the audio clock; handles pause, seek and playback speed (e.g. 1.5x).
Inputs / Outputs	Input: audio playback position from [27]. Output: presentation timing for [26].
Fine details	Audio is usually the master clock (people notice audio glitches more). Video frames that are too late are dropped; too early frames wait. Target drift is within a few tens of milliseconds.
Why it matters	Out-of-sync video looks broken; this keeps playback feeling professional.

4.6 Layer 4: C++ core engine, media pipeline group

[23] Demuxer

What it is	Splits each downloaded segment into separate streams.
Analogy	Sorting mixed mail into letters, parcels and magazines.
What it does	Reads container formats (fMP4 / CMAF, MPEG-TS) and outputs separate compressed packets for video, audio and subtitles, each with a timestamp.
Inputs / Outputs	Input: segment bytes from [18]. Output: timestamped packets to the buffer [24].
Fine details	Handles timestamp discontinuities (e.g. ad breaks), extracts codec settings (H.264/HEVC, AAC) and encryption info per sample.
Why it matters	Correct timestamps here are the foundation for smooth playback and seeking.

[24] Buffer Manager

What it is	Stores media that has been downloaded but not yet played.
Analogy	A water tank: download fills it, playback drains it. If it empties, the tap stops (buffering).
What it does	Keeps separate queues for audio, video and subtitles. Tracks how many seconds are buffered and signals “low” and “full” thresholds. Clears or trims data on seek or quality switch.
Inputs / Outputs	Input: packets from [23]. Output: packets to the decoder [25]; buffer-level reports to [20] and [14].
Fine details	Uses memory limits (in bytes as well as seconds) so the SDK never uses too much RAM on older devices; uses lock-free or carefully locked ring buffers between threads.
Why it matters	The right buffer size balances fast start, few stalls and low memory use.

[25] Decoder Wrapper

What it is	A thin layer around Apple’s hardware decoders.
Analogy	A translator who turns a compressed message back into full sentences.
What it does	Feeds compressed packets to VideoToolbox [29] and AudioToolbox [30] and gets back decoded frames and audio samples.
Inputs / Outputs	Input: packets from [24]. Output: decoded video frames to [26]; audio samples to [27].
Fine details	Uses hardware decoding to save battery; recreates the decoder session if the app returns from background or the codec changes; reports decode errors and can fall back to a lower rendition.
Why it matters	Hardware decoding means smooth 4K/HDR and long battery life.

[26] Video Renderer

What it is	Draws decoded frames onto the screen at the right time.
Analogy	A projectionist showing each film frame at exactly the right moment.
What it does	Takes frames and timing from [22], shows them in the player view using AVSampleBufferDisplayLayer or Metal [31], and handles scaling (fit/fill), rotation and HDR.
Inputs / Outputs	Input: frames and clock time. Output: pixels on screen; dropped-frame counts to [17].
Fine details	Supports Picture-in-Picture and AirPlay through Apple APIs; partners can overlay their own controls on top of the view.
Why it matters	This is what the viewer actually sees.

[27] Audio Output

What it is	Sends decoded sound to speakers or headphones.
Analogy	The sound system in a cinema.

What it does	Plays audio, applies volume and mute, handles headphone unplugging, phone calls and other interruptions, and reports its play position to the clock [22].
Inputs / Outputs	Input: audio samples. Output: sound; current audio time to [22].
Fine details	Configures the audio session for playback and background audio; pauses automatically on interruption and resumes when appropriate (following Apple guidelines).
Why it matters	Correct interruption handling is required for App Store approval and good user experience.

4.7 Layer 5: Apple platform frameworks

These are not built by the team. They are system libraries that Apple provides. The SDK uses them so it gets hardware speed, battery efficiency and platform features for free.

#	Framework	Simple meaning	Used by
[28]	AVFoundation / CoreMedia	Apple's base media toolkit: time values (CMTime), sample buffers, audio sessions, display layers.	[22], [25], [26], [27]
[29]	VideoToolbox	Hardware video decoder chip access (H.264, HEVC). Fast and battery-friendly.	[25]
[30]	AudioToolbox / AVAudio	Decodes audio (AAC etc.) and plays it through the speaker.	[25], [27]
[31]	Metal / CoreAnimation	Graphics systems for drawing frames on screen efficiently.	[26]
[32]	URLSession / Network	Secure internet downloads (HTTPS), including background transfers.	[18]
[33]	FairPlay Streaming	Apple's DRM system that safely handles decryption keys.	[21]

4.8 Layer 6: Outside services

#	Service	Simple meaning	Owned by
[34]	CDN / Origin Server	Stores manifests and video segments on servers worldwide so downloads are fast.	Each partner (their content)
[35]	DRM License Server	Checks if the user is allowed to watch and returns the decryption key.	Each partner or their DRM vendor
[36]	Telemetry Backend	Collects anonymous playback quality numbers and shows them on dashboards.	SDK team and/or partners

4.9 Delivery and governance components (Figure 2)

[37] Partner Requirements Alignment

What it is	A structured process for collecting, comparing and agreeing on what all four partners need.
Analogy	Planning a group trip: first agree on the destination, dates and budget before booking.
What it does	Kick-off workshops with each partner's engineering and product teams. One shared requirements matrix: rows = features (DRM, offline, live, analytics...), columns = partners, cells = must-have / nice-to-have / not needed. Finding the common core all partners need versus partner-specific extensions handled via config [16] or hooks.
Inputs / Outputs	Input: partner needs, deadlines, constraints. Output: prioritised, signed-off roadmap.
Fine details	Regular syncs (e.g. bi-weekly), a shared issue tracker, and named contacts on both sides; disagreements resolved by data (usage, deadlines) rather than whoever shouts loudest.
Why it matters	Removes late surprises and rework, one of the three drivers of the 45% saving.

[38] API Design Review

What it is	A written proposal (RFC) plus draft header files reviewed before any code is written.
Analogy	Reviewing blueprints before building a house; changes on paper are cheap.
What it does	Describes each new API, example usage, error cases and alternatives; partners review and comment early.
Inputs / Outputs	Input: roadmap items from [37]. Output: approved public headers.
Fine details	Checklist: consistent naming, Swift-friendliness, thread-safety, nullability, error codes, versioning impact, documentation.
Why it matters	Partners shaped the API they would use, so it fit their apps first time.

[39] Build and Continuous Integration (CI)

What it is	An automated system that builds and tests the SDK on every code change.
Analogy	A quality-control line in a factory that checks every product.
What it does	Unit tests for each engine component; integration tests playing real streams. Sanitizers (Address/Thread Sanitizer) to catch memory and threading bugs. Performance checks: start-up time, CPU, memory and battery against limits. Public API diff check (see [9]) and building the reference players [5]/[6].
Inputs / Outputs	Input: code changes. Output: pass/fail result, test reports, build artifacts.
Fine details	Runs on multiple OS versions and device types, including real devices for DRM and hardware decode.
Why it matters	Partners received fewer bugs, so less of their time went to debugging the SDK.

[40] Versioned Release (SemVer + XCFramework)

What it is	Packaging and publishing each SDK version.
-------------------	--

Analogy	Shipping a product with a clear model number and a sealed box.
What it does	Builds one XCFramework (devices + simulators + Mac), tags it with a version like 2.3.0, and publishes it through Swift Package Manager and CocoaPods along with the matching reference player.
Inputs / Outputs	Input: a green CI build. Output: a downloadable, signed, versioned SDK.
Fine details	Release channels: beta (early partner testing) and stable; long-term support for the previous major version; dSYM debug symbols shipped so partner crash reports are readable.
Why it matters	Predictable, safe upgrades, the “versioned APIs” part of the resume line.

[41] Docs, Changelog and Migration Guides

What it is	All written help that ships with each release.
Analogy	The instruction manual plus a “what’s new” leaflet.
What it does	Generated API reference (from header comments), getting-started guide, how-to recipes (DRM, offline, PiP), changelog, and step-by-step migration guides between major versions.
Inputs / Outputs	Input: header comments and release notes. Output: a documentation site/package.
Fine details	Every code sample in docs is compiled in CI so examples never go stale.
Why it matters	Partners found answers themselves instead of waiting on support.

[42] Partner Integration and Feedback

What it is	Supporting partners while they integrate, and collecting what they learn.
Analogy	A follow-up call after delivery to ask “How is it working for you?”
What it does	Shared chat channel, office hours, integration checklist, and tracking each partner’s integration milestones; issues and feature requests feed back into [37].

Inputs / Outputs	Input: partner questions, bug reports, metrics. Output: fixes, improved docs, next roadmap items.
Fine details	Measured integration time per partner (start date to production launch) to prove the 45% improvement.
Why it matters	Closes the loop so each release is easier to integrate than the last.

5. Workflows, Step by Step

This section follows the project from planning to a viewer pressing play. Numbers in square brackets refer to components in Section 4.

5.1 Workflow A: Aligning requirements with partners

1. **Kick-off:** meet each of the 4 partners separately to understand their app, audience, content type and launch date. [37]
2. **Collect needs:** list required features (DRM, live, offline, subtitles, analytics, PiP, AirPlay) and constraints (minimum iOS version, app size limits).
3. **Build the requirements matrix:** one table comparing all partners side by side, marking each item must-have / nice-to-have / not needed.
4. **Find the shared core:** features most partners need go into the main SDK; unusual needs become configuration options [16] or extension hooks rather than custom code.
5. **Prioritise:** order features by how many partners need them and how soon.
6. **Agree and sign off:** share the roadmap with all partners; everyone knows what arrives in which version.
7. **Keep in sync:** regular check-ins and a shared tracker so changes are agreed, not discovered late.

Result: no partner is surprised later, and the team builds each feature once for everyone.

5.2 Workflow B: Designing and versioning an API

1. **Write an RFC:** a short document describing the new capability, example code, error cases and alternatives considered. [38]
2. **Draft the header:** write the actual public Objective-C header and C++ interface first, with doc comments. [7] [8]
3. **Review:** internal engineers and partner engineers review for clarity, naming, Swift usage, thread-safety and future-proofing.
4. **Decide the version impact:** new additions mean a MINOR bump; behaviour fixes mean a PATCH; anything that breaks existing code waits for a MAJOR release. [9]
5. **Deprecate, don't delete:** if an old method is replaced, mark it deprecated with the new name; keep it working via a shim for at least one major version. [9]
6. **Implement behind a flag** if risky, so partners can trial it first. [16]
7. **Lock the API:** CI's API-diff check stops accidental breaking changes. [39]

5.3 Workflow C: Build, test and release

1. **Engineer pushes code** to the repository.
2. **CI builds** the SDK for iOS devices, simulators and macOS. [39]
3. **Automated tests run:** unit tests, playback tests with sample streams (clear and DRM), sanitizers, performance limits and API-diff check.
4. **Reference players are built** using only the public API, proving nothing is missing. [5] [6]
5. **Release candidate:** version number assigned (e.g. 2.4.0), XCFramework packaged and signed, debug symbols attached. [40]
6. **Beta to partners:** partners test it in their apps for a short window.
7. **Stable release:** published via Swift Package Manager / CocoaPods with changelog and migration notes. [41]

5.4 Workflow D: How a partner integrates the SDK

This is the workflow that became 45% faster.

1. **Add the SDK** to the app with one Swift Package Manager line or CocoaPods entry. [40]
2. **Run the reference player** to see a working example on their own device in minutes. [5] [6]
3. **Copy the basic pattern:** create a player, attach the player view, load a URL, and press play.
4. **Plug in their specifics:** DRM token hook [21], custom HTTP headers [18], analytics hook [17], configuration options [16].
5. **Build their own UI** on top of the player view, listening to events [10] to update buttons and progress bars.
6. **Test with the debug overlay** and logs to verify bitrate, buffering and errors. [17]
7. **Launch**, then upgrade to new SDK versions using the changelog and migration guides. [9] [41]

Step	Before (no SDK structure)	After (this project)
Understanding needs	Requirements discovered mid-integration; rework	Agreed up front in the requirements matrix
Getting started	Read raw code, trial and error	Run reference player, copy working pattern
DRM setup	Built from scratch by each partner	Built-in flow with one hook for auth
Debugging	Guesswork, vague errors	Clear error codes, logs and debug overlay

Step	Before (no SDK structure)	After (this project)
Upgrading	Breaking changes, risky	SemVer, deprecations, migration guides
Overall integration time	Baseline (100%)	About 55% of baseline (45% faster)

5.5 Workflow E: Runtime playback, end to end

What happens inside the SDK from the moment the app creates a player to the moment the video is closed.

Stage 1: Set-up

1. The app creates an `MPPlayer` and an `MPPlayerView` and sets itself as the delegate. [7]
2. The bridge creates the matching C++ player and links their lifetimes. [11] [13]
3. The Player Controller starts in the **Idle** state and reads configuration. [14] [16]

Stage 2: Loading

1. The app calls `load(url)`. The controller moves to **Loading** and tells the Source Loader to fetch the manifest. [14] [18]
2. The Source Loader downloads the manifest from the CDN over HTTPS. [32] [34]
3. The Manifest Parser reads it: quality levels, audio/subtitle tracks, segment list, DRM info. [19]
4. If the content is encrypted, the DRM Manager requests a key: FairPlay builds a request, the partner hook adds an auth token, the license server returns the key. [21] [33] [35]

Stage 3: Starting playback

1. The ABR Controller picks a safe starting quality (often medium) for a fast start. [20]
2. The Source Loader downloads the first few segments and measures download speed. [18]
3. The Demuxer splits each segment into audio, video and subtitle packets with timestamps. [23]
4. Packets fill the Buffer. When enough is buffered (e.g. 2–3 seconds), the controller moves to **Ready** and notifies the app. [24] [14] [10]
5. The app calls `play()`. The controller moves to **Playing**. [14]

Stage 4: Steady playback loop (repeats continuously)

1. The Decoder Wrapper takes packets from the buffer and uses hardware to decode them. [25] [29] [30]
2. Audio Output plays sound and reports its position to the Master Clock. [27] [22]

3. The Video Renderer shows each frame exactly when the clock says, dropping late frames. [26] [22]
4. Meanwhile the Source Loader keeps downloading ahead to keep the buffer near its target. [18] [24]
5. The ABR Controller re-evaluates quality before each new segment using speed and buffer level. [20]
6. Time updates and bitrate-change events flow up to the app on the main thread. [15] [12]
7. Analytics records QoE metrics throughout. [17]

Stage 5: Things that can happen during playback

- **Network slows down:** ABR switches to a lower quality; if the buffer still empties, the state becomes **Buffering**, then returns to Playing once refilled. [20] [24] [14]
- **User seeks:** state becomes **Seeking**; downloads are cancelled, the buffer is cleared, the loader fetches the segment at the new time, decoding restarts, then back to Playing. [14] [18] [24]
- **User pauses:** the clock stops, rendering and audio pause; downloading may continue until the buffer is full. [22]
- **Phone call or headphones unplugged:** audio output pauses playback and reports it to the app. [27]
- **App goes to background:** video rendering stops; audio can continue if the partner enabled background audio; decoder sessions are recreated on return. [25] [26]
- **Error:** see Workflow F (Section 5.7).

Stage 6: Ending and clean-up

1. At the end of the stream the controller moves to **Ended** and notifies the app. [14] [10]
2. When the app releases the player, the bridge shuts down the C++ engine: downloads cancelled, decoders closed, memory freed. [13]
3. Final analytics summary for the session is sent. [17] [36]

5.6 Player states and allowed moves

State	Simple meaning	Can move to
Idle	Player exists, nothing loaded.	Loading
Loading	Fetching manifest, keys and first segments.	Ready, Failed
Ready	Enough data to start; waiting for play().	Playing, Seeking, Failed

State	Simple meaning	Can move to
Playing	Video and audio are running.	Paused, Buffering, Seeking, Ended, Failed
Paused	Stopped by the user; position kept.	Playing, Seeking, Failed
Buffering	Ran out of data; waiting to refill.	Playing, Paused, Seeking, Failed
Seeking	Jumping to a new time position.	Playing, Paused, Buffering, Failed
Ended	Reached the end of the content.	Seeking (replay), Idle
Failed	A fatal error happened.	Idle (after reset or new load)

5.7 Workflow F: Error handling

1. **Detect:** a component notices a problem (e.g. download failed, decode failed, license denied) and posts it on the Event Bus. [15]
2. **Classify:** the error is mapped to a documented code and marked recoverable or fatal. [10]
3. **Try to recover automatically:** retry the download with backoff, switch to a backup CDN, drop to a lower quality, or recreate the decoder. [18] [20] [25]
4. **Tell the app:** if recovery works, a warning event is sent; if not, the state becomes **Failed** and the app receives the error with its cause chain. [14] [12]
5. **Record:** the error and context are logged and counted in analytics for later diagnosis. [17]
6. **App decides:** show a friendly message, offer retry, or fall back to other content.

5.8 Workflow G: Upgrading to a new SDK version

1. Partner reads the changelog to see what changed. [41]
2. For a MINOR or PATCH update, they update the version number and rebuild; nothing should break. [40]
3. For a MAJOR update, they follow the migration guide; Xcode deprecation warnings point to each replacement method. [9]
4. They compare behaviour with the matching reference player version. [5] [6]
5. Feedback or issues go back to the SDK team. [42]

6. Results and Why They Happened

Outcome	How the design caused it
Adopted by 4 third-party partners	A shared core plus configuration and hooks meant one SDK served four different products without custom forks.
45% faster integration	Early requirement alignment (less rework), reference players (working starting point), versioned APIs (safe upgrades), and clear errors and docs (faster debugging).
Stable upgrades	SemVer, deprecation shims and an automated API-diff check in CI.
High playback quality	Hardware decoding, ABR, a tuned buffer and an accurate A/V clock.
Portability	C++ core separated from Apple-specific layers via the bridge, ready for other platforms.

6.1 How to explain this in an interview (30 seconds)

“I led the design of a video playback SDK for Apple platforms. The engine was C++ for performance and portability, exposed through a clean Objective-C API that Swift apps could use, with an Objective-C++ bridge in between. Four external partners adopted it. Beyond the architecture, I focused on making integration easy: we aligned requirements with all partners up front, shipped semantically versioned APIs with deprecation paths, and provided full reference players. That cut partner integration time by about 45%.”